LoRa Messager — les fonctions LoRa / Bluetooth

Deux fichiers portent toute la partie radio : `src/ble.js` (liaison Bluetooth) et `src/mesh.js` (protocole Meshtastic). La colonne de droite indique d'où vient chaque fonction dans le dépôt `github.com/meshtastic/web`.

Fonction	Ce qu'elle fait	Origine dans le dépôt
1. Connexion Bluetooth — `ble.js`		
`requestAndConnect()`	Ouvre le sélecteur Bluetooth (radios Meshtastic uniquement) puis se connecte.	`transport-web-bluetooth/src/transport.ts` → `create()`
`connect()`	Récupère le service et les 3 caractéristiques (`toRadio`, `fromRadio`, `fromNum`), active les notifications.	`transport.ts` → `prepareConnection()`
`connectGatt()`	Connexion GATT, avec une seconde tentative si la radio sort de veille.	`transport.ts` → `connectGatt()`
`gatt()`	File d'attente : une seule opération Bluetooth à la fois.	Ajout
`disconnect()`	Coupe la liaison.	`transport.ts` → `disconnect()`
2. Échange d'octets avec la radio — `ble.js`		
`write(bytes)`	Écrit un message dans `toRadio`.	`transport.ts` → `toDevice` (WritableStream)
`readAll()`	Lit `fromRadio` jusqu'à vide. Appelée à chaque notification `fromNum`.	`transport.ts` → `readFromRadio()`
3. Poignée de main — `mesh.js`		
`configure()`	Demande la config à la radio en 2 étapes (`wantConfigId` 69420 puis 69421). Obligatoire avant d'envoyer.	`sdk/src/core/client/MeshClient.ts` → `configure()` ; nonces dans `handshake.ts`
`startStage()`, `onConfigComplete()`	Envoient chaque étape et passent à la suivante à la réception de `configCompleteId`.	`MeshClient.ts` → `startHandshakeStage()`, `wireHandshake()`
`armHandshakeTimer()`, `failHandshake()`	Échec si la radio se tait 15 s.	`MeshClient.ts` (mêmes noms)
4. Maintien de la session — `mesh.js`		
`heartbeat()`, `startHeartbeat()`	Signal de vie toutes les 30 s.	`MeshClient.ts` → `heartbeat()`, `setHeartbeatInterval()`
`close()`	Arrête tout à la déconnexion.	`MeshClient.ts` → `disconnect()`
5. Envoi d'un message — `mesh.js`		
`sendText(text, {to, channel})`	Encode le texte (228 octets max) dans un `MeshPacket` de type `TEXT_MESSAGE_APP` et l'envoie.	`sdk/src/features/chat/application/SendTextUseCase.ts` + `MeshClient.ts` → `sendPacket()`
`sendToRadio()`	Encode un `ToRadio` (protobuf) et l'écrit.	`MeshClient.ts` → `sendRaw()`

`generatePacketId()`	Identifiant aléatoire du message.	`sdk/src/core/identifiers/PacketId.ts`
6. Accusés de réception — `mesh.js`		
`waitForAck()`	Attend l'accusé du message (60 s max).	`sdk/src/core/queue/Queue.ts` → `push()`
`settleAck()`	Paquet `ROUTING_APP` reçu : succès si `NONE`, sinon erreur (ex. `MAX_RETRANSMIT` = personne n'a relayé).	`Queue.ts` → `processAck()` / `processError()`
7. Réception — `mesh.js`		
`handleFromRadio(bytes)`	Décode un `FromRadio` et trie : infos du nœud, canaux, config LoRa, fin de config, paquet.	`sdk/src/core/packet-codec/decodePacket.ts` → `decodePacket()`
`handleMeshPacket(packet)`	Écarte les doublons, puis : texte reçu, accusé, ou nom d'un nœud.	`decodePacket.ts` → `handleMeshPacket()`
8. Partage de canal — `mesh.js`		
`channelShareUrl(indices, {add})`	Crée le lien `meshtastic.org/e/#…` (nom + clé des canaux + config LoRa) affiché en QR code.	`apps/web/src/core/utils/channelUrl.ts` → `channelSetToUrl()`

Le format des messages (`ToRadio`, `FromRadio`, `MeshPacket`…) vient du paquet `@meshtastic/protobufs`. En Bluetooth, les messages ne portent pas l'en-tête `0x94 0xC3` : celui-ci sert uniquement à l'USB (`toDevice.ts` / `fromDevice.ts`).